

RDI Platform

A Full-Stack Web Application for Collaborative Innovation Ecosystems

Abstract

Author(s) Parth Mangukiya	Publication type Final assignment	Completion year 2026
	Number of pages 10	
Title of the thesis RDI Platform A Full-Stack Web Application for Collaborative Innovation Ecosystems		
Degree Bachelor of Engineering, Industrial Information Technology		
Name, title and organisation of the client Mira Vorne, Lab University Of applied sciences.		
Abstract The Research, Development, and Innovation (RDI) Platform is a full-stack web application designed to facilitate collaboration between students, mentors, companies, and expert reviewers within a structured innovation ecosystem. The platform enables users to create, manage, and evaluate research and development projects while supporting team collaboration and real-world industry engagement. Built using modern technologies such as Node.js, Express, MongoDB, and React, the system implements role-based access control to provide tailored functionalities for different user types. The platform incorporates key features including project lifecycle management, team collaboration tools, opportunity posting for companies, and a multi-dimensional review and rating system. Authentication is handled through JSON Web Tokens (JWT), ensuring secure and scalable access control. Additionally, the system supports RESTful APIs for seamless frontend-backend communication and dynamic data visualization. Overall, the RDI Platform aims to bridge the gap between academia and industry by fostering innovation, improving project evaluation mechanisms, and enhancing collaborative learning experiences.		
Keywords RDI Platform, Full-Stack Development, Collaboration System, Project Management, Innovation Ecosystem, Role-Based Access Control, Web Application		

Contents

1	Project Overview.....	Error! Bookmark not defined.
2	Architecture	Error! Bookmark not defined.
3	Date Model	2
4	Authentication	4
5	API Reference	5
6	Frontend Routing	7
7	Review & Rating system	7
8	Know Issues & Notes.....	8
9	Screenshot	9

1 Project Overview

The RDI Platform (Research, Development & Innovation) is a full-stack web application that connects students, mentors, companies, and expert reviewers in a structured innovation ecosystem. Students create and manage research/development projects, invite team members, and receive scored feedback from multiple reviewer types. Companies post internship and collaboration opportunities that students can apply to. Admins and mentors oversee the full platform.

I - Core Features

- Role-based access for five distinct user types: student, mentor, admin, company, expert
- Project lifecycle management: Draft → Open for Members → In Progress → Submitted → Completed
- Team collaboration: invite members by email, manage join applications
- Multi-dimensional project rating system with per-role scoring criteria
- Company opportunity board with student application tracking
- Admin dashboard for user, project, team, and opportunity oversight

II - Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	JavaScript server runtime
Framework	Express	HTTP server and routing
Database	MongoDB (Mongoose)	Document store, ODM
Authentication	JSON Web Token (JWT)	Stateless auth tokens
Password Hashing	Bcrypt	Secure password storage
Frontend	React	UI component framework
Build Tool	Vite	Dev server and bundler
Styling	Tailwind CSS	Utility-first CSS
Routing (FE)	React Router DOM	Client-side routing
HTTP Client	Axios	API calls from frontend

2 Architecture

I - High-Level Structure

Layer	Location	Description
Frontend (SPA)	frontend/src/	React app served by Vite; role-based route guards
Backend API	backend/src/	REST API; controllers, routes, middle-ware, models
Database	MongoDB Atlas / local	Collections: Users, Projects, Opportunities, Applications, Reviews
Auth	JWT Bearer tokens	Token issued on login, verified on every protected route

3 Data Models

I – User

Field	Type	Equired	Notes
Name	String	Yes	Trimmed
Email	String	Yes	Unique, lowercase, trimmed
Password	String	Yes	bcrypt hashed (salt rounds: 10)
Role	String (enum)	No	student mentor admin company expert. Default: student
createdAt / updatedAt	Date	Auto	Mongoose timestamps

II – Project

Field	Type	Notes
Title	String	Required
shortDescription	String	Required — summary shown in listings
Description	String	Required — full detail
Category	String	Required
Skills	[String]	Array of required skill tags
duration / teamSize / goals	String / Number / String	Optional project metadata
startDate / endDate / deadline	Date	Optional schedule fields
openForMembers	Boolean	Default true — controls join availability
Status	String (enum)	Draft Open for Members In Progress Submitted Completed Closed
projectType	String (enum)	individual group. Default: individual
CreatedBy	ObjectId → User	Required — project owner
Members	[{ user, role }]	role: leader member. No separate _id
pendingMembers	[{ email }]	Invited but not yet registered users
ratingSummary	Object	compositeScore, mentorAverage, peerAverage, companyAverage, expertAverage, totalRatings, isVerified

III – Opportunity

Field	Type	Notes
Title	String	Required
companyName	String	Required
description	String	Required
skillsRequired	String	Default []
Type	String (enum)	internship project job collaboration. Default: internship
location	String	Default "
Deadline	Date	Optional
createdBy	ObjectId → User	Required — company user who posted
Applicants	[[student, status, appliedAt]]	status: pending shortlisted rejected accepted

IV – Application

Records a student's request to join a project team. A compound unique index on { projectId, applicantId } prevents duplicate applications.

Field	Type	Notes
projectId	ObjectId → Project	Required
applicantId	ObjectId → User	Required
Message	String	Optional cover note
Status	String (enum)	pending accepted rejected. Default: pending

V – Review

Stores a reviewer's scored assessment of a project. A compound unique index on { project, reviewer, reviewerRole } prevents duplicate review assignments per role.

Field	Type	Notes
Project	ObjectId → Project	Required
Reviewer	ObjectId → User	Required
reviewerRole	String (enum)	mentor peer company expert
Criteria	[[key, label, score]]	score 1–5; populated from role template on submit
Feedback	String	Free-text comment
averageScore	Number	Calculated average of criteria scores
Status	String (enum)	assigned completed
submittedAt	Date	Set on submission

Review Criteria Templates (per role)

Reviewer Role	Criterion Keys	Labels
Mentor	learningProgress, completionQuality, academicValue	Learning progress, Completion quality, Academic value
Peer	teamwork, communication, reliability	Teamwork, Communication, Reliability
Company	commercialRelevance, practicalUsefulness, professionalQuality	Commercial relevance, Practical usefulness, Professional quality

Expert	technicalQuality, innovation, accuracy	Technical quality, Innovation, Accuracy
---------------	--	---

4 Authentication

I - How It Works

The platform uses stateless JWT authentication. On login, the server signs a token containing the user's MongoDB _id with a 7-day expiry. The client stores the token (typically in localStorage) and sends it as a Bearer token in the Authorization header on every subsequent request.

```
// Token generation (authController.js)

const generateToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET, {
  expiresIn: '7d' });

// protect middleware (authMiddleware.js)

const token = req.headers.authorization.split(' ')[1];

const decoded = jwt.verify(token, process.env.JWT_SECRET);

req.user = await User.findById(decoded.id).select('-password');
```

II - Roles & Permissions

Role	Access Level	Key Capabilities
Student	Protected routes under /student	Create projects, apply to join teams, apply to opportunities, submit peer reviews, view own score profile
Mentor	Protected routes under /admin (shared)	View all users/projects/teams, assign reviewers, submit mentor reviews, oversee platform
Admin	Protected routes under /admin + ManageUsers + Settings	All mentor capabilities plus manage users and platform settings
Company	Protected routes under /company\	Post opportunities, view own opportunities, manage applicant status, view submitted projects
expert	Protected routes under /expert	View assigned review tasks, submit expert reviews, view review history\

III – Middleware

Middleware	Function	Usage
Protect	Verifies JWT, attaches req.user	Applied to every protected route
authorizeRoles(...roles)	Checks req.user.role is in allowed list	Applied after protect on role-restricted routes

5 API Reference

Base URL: <http://localhost:5000>. All protected endpoints require Authorization: Bearer <token>. All request/response bodies use JSON.

I - Authentication — /api/auth

Method	Endpoint	Auth	Description	Key Fields
POST	/api/auth/register	None	Create a new user account	name, email, password, role
POST	/api/auth/login	None	Login and receive JWT token	email, password
GET	/api/auth/me	Bearer	Get current user profile	----

POST /api/auth/register — Request body

```
{
  "name": "parth mangukiya",
  "email": "parthmangukiya@gmail.com",
  "password": "secret123",
  "role": "student"
}
// 201 Response
{
  "message": "User registered successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "...", "name": "Jane Smith", "email": "jane@example.com", "role": "student" }
}
```

II - Projects — /api/projects

Method	Endpoint	Role	Description
GET	/api/projects	Any	Get all projects (filtered by role)
GET	/api/projects/:id	Any	Get project by ID with full details
POST	/api/projects	Any	Create a new project
POST	/api/projects/:id/apply	student	Apply to join a project team
GET	/api/projects/:id/applications	Owner / admin	List all join applications for a project
PUT	/api/projects/:id/application-status	Owner	Accept or reject a join application
DELETE	/api/projects/:id/members/:userId	Owner	Remove a member from the project
POST	/api/projects/:id/leave	Member	Leave a project team
PUT	/api/projects/:id/complete	Owner	Mark project as completed

III - Opportunities — /api/opportunities

Method	Endpoint	Role	Description
GET	/api/opportunities	Any	Get all opportunities

GET	/api/opportunities/my	Company	Get opportunities created by the logged-in company
POST	/api/opportunities	company	Create a new opportunity posting
POST	/api/opportunities/:id/apply	Student	Apply to an opportunity
PUT	/api/opportunities/:id/applicant-status	company	Update applicant status (shortlist/accept/reject)

IV - Reviews — /api/reviews

Method	Endpoint	Description
POST	/api/reviews	Create a single review assignment (manual)
GET	/api/reviews/my	Get all reviews assigned to the current user
GET	/api/reviews/candidates	Get all mentor/expert/company users (for assigning reviewers)
PUT	/api/reviews/:id/submit	Submit scores and feedback for a review
GET	/api/reviews/project/:projectId/summary	Get rating summary and completed reviews for a project
GET	/api/reviews/project/:projectId	Get all reviews for a project
POST	/api/reviews/project/:projectId/assign	Bulk-assign multiple reviewers to a project

PUT /api/reviews/:id/submit — Request bod

```
{
  "scores": {
    "technicalQuality": 4,
    "innovation": 5,
    "accuracy": 3
  },
  "feedback": "Strong technical foundation with creative approach."
}
```

V - Admin — /api/admin

All admin endpoints require Bearer token and role: admin OR mentor.

Method	Endpoint	Description
GET	/api/admin/users	Get all users (passwords excluded)
GET	/api/admin/projects	Get all projects with populated creator and members
GET	/api/admin/teams	Get all projects formatted as team listings
GET	/api/admin/opportunities	Get all opportunities with populated creator

6. Frontend Routing

React Router v7 handles all client-side routing. ProtectedRoute wraps role-sensitive layouts, redirecting unauthenticated users to /login. Role mismatches also redirect. All unmatched paths redirect to /login.

Path	Component	Auth / Role Required	Description
/login	Login	None	Login form
/register	Register	None	Registration form
/student/dashboard	DashboardHome	student	Student home with project overview
/student/projects	MyProjects	student	List of own and joined projects
/student/projects/:id	ProjectDetails	student	Full project detail + team management
/student/create	CreateProject	student	New project creation form
/student/team	TeamPage	student	Team member view
/student/reviews	ReviewsPage	student	Assigned peer reviews
/student/score-profile	StudentProfileScore	student	Personal rating/score summary
/student/opportunities	OpportunitiesPage	student	Browse and apply to opportunities
/admin/dashboard	AdminDashboardHome	admin / mentor	Platform stats overview
/admin/projects / reviews	ReviewProjects	admin / mentor	Review and manage all projects
/admin/teams	TeamsPage	admin / mentor	View all teams
/admin/opportunities	AdminOpportunitiesPage	admin / mentor	View all opportunities
/admin/users	ManageUsers	admin only	Manage user accounts
/admin/settings	SettingsPage	admin only	Platform settings
/company/dashboard	CompanyDashboardHome	company	Company home
/company/opportunities	CompanyOpportunities	company	Manage own opportunity listings
/company/projects	CompanyProjects	company	View submitted student projects
/company/applicants	CompanyApplicants	company	View all opportunity applicants
/expert/dashboard	ExpertDashboardHome	expert	Expert home
/expert/reviews	ExpertReviews	expert	Assigned review tasks
/expert/history	ExpertHistory	expert	Completed review history

7 Review & Rating System

The rating system is one of the platform's core differentiators. Each completed project receives reviews from up to four reviewer types. Each reviewer type uses a fixed 3-criterion template. All scores are on a 1–5 scale.

I - Review Flow

1. A project owner or admin calls POST /api/reviews/project/:id/assign with a list of reviewers and roles.
2. Each reviewer sees the assignment in their dashboard under GET /api/reviews/my.
3. The reviewer submits scores and feedback via PUT /api/reviews/:id/submit.
4. On submission, recalculateProjectRatingSummary() runs automatically and updates the project's ratingSummary fields.

II - Composite Score Calculation

The composite score is the average of all completed review averageScore values. Individual role averages (mentorAverage, peerAverage, etc.) are stored separately. The project is flagged isVerified: true once at least one review is completed

III - Who Can Review What

User Role	Review Role Assigned	Criteria Evaluated
Mentor	Mentor	Learning progress, Completion quality, Academic value
Student	Peer	Teamwork, Communication, Reliability
Company	Company	Commercial relevance, Practical usefulness, Professional quality
Expert	Expert	Technical quality, Innovation, Accuracy
Admin	Any	Can assign or submit any review type

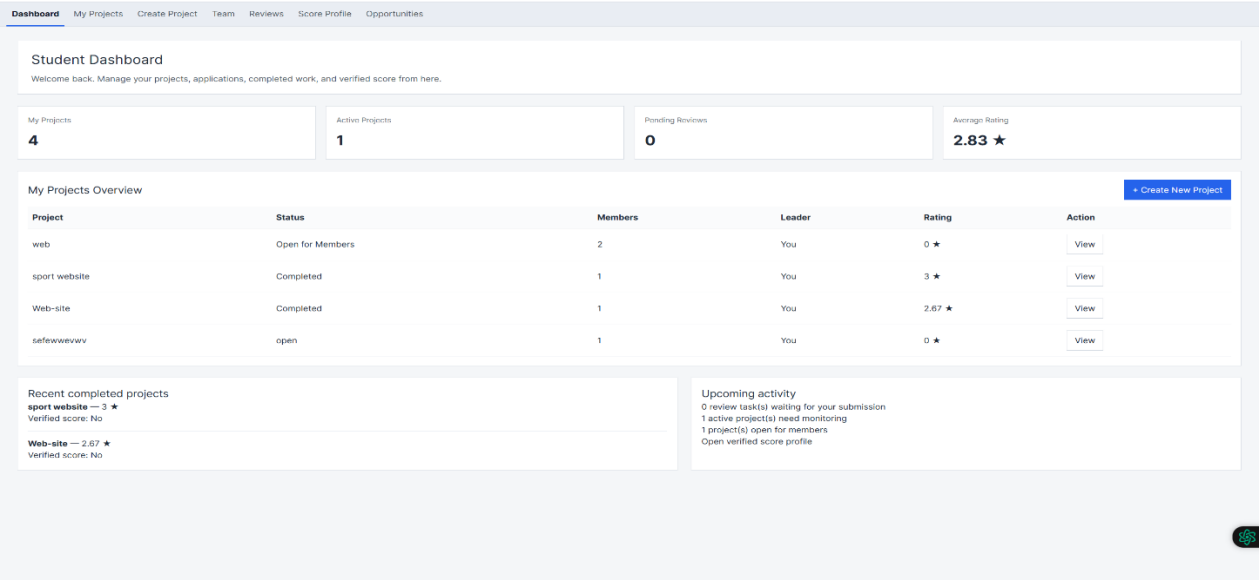
8 - Known Issues & Notes

Issue	Location	Description
Duplicate route import	server.js	adminRoutes is imported and mounted twice. The second mount overrides the first — remove one import and one app.use() call.
Commented-out code in routes	projectRoutes.js	A large React component (ProtectedRoute) is commented out at the bottom of a backend route file. Should be removed.
No .env validation	server.js	If MONGO_URI or JWT_SECRET are missing, the server fails silently. Add a startup check.
No rate limiting	server.js	Auth endpoints have no rate limiting. Consider adding express-rate-limit to /api/auth.
Token stored in localStorage	Frontend (assumed)	JWTs in localStorage are vulnerable to XSS. Consider httpOnly cookies for production.

9 Screenshots

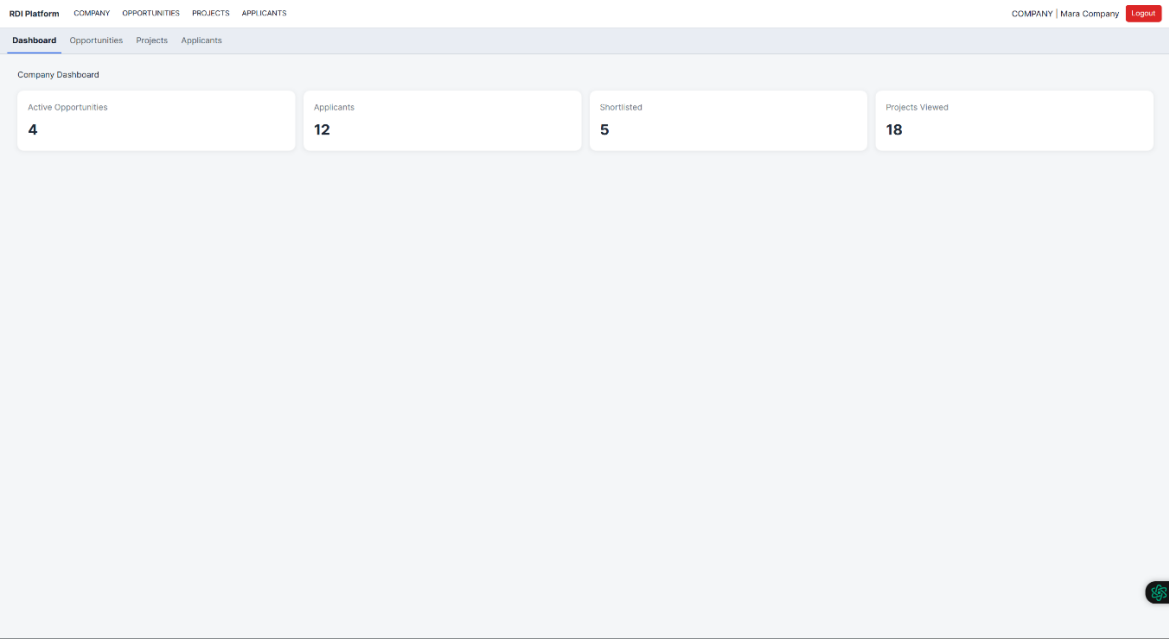
I - Student Dashboard

The Student Dashboard provides a personal overview of projects, pending reviews, average rating, and upcoming activity. Students can navigate to projects, create new ones, view team members, submit peer reviews, check their score profile, and browse company opportunities.



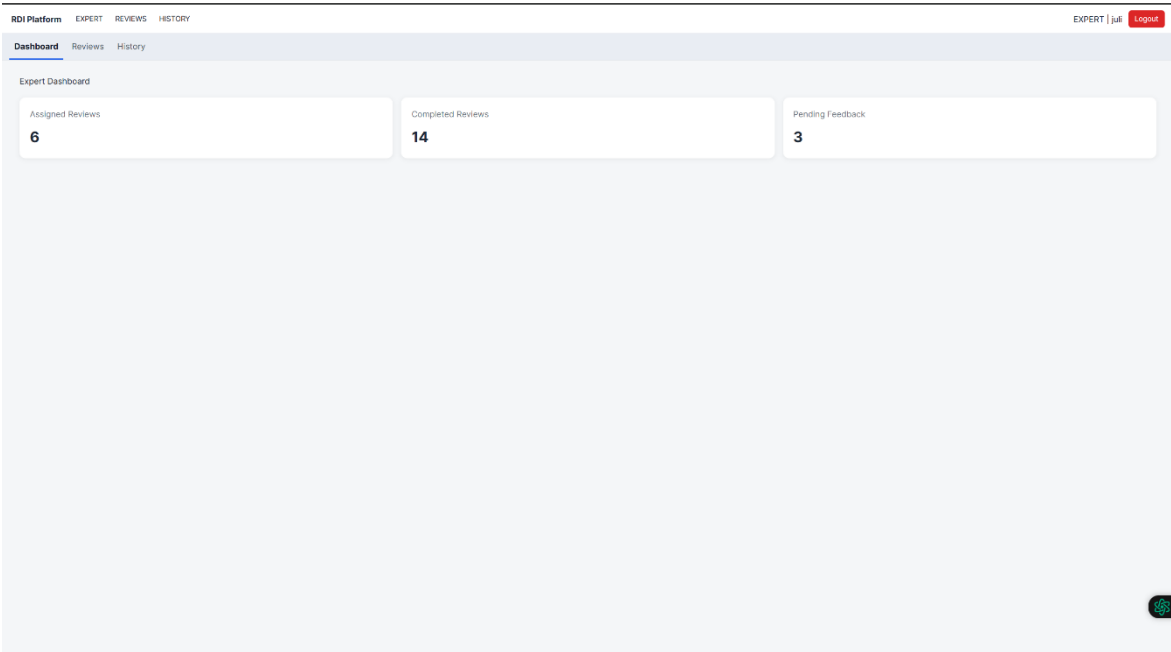
II - Company Dashboard

The Company Dashboard gives a quick overview of active opportunities, total applicants, shortlisted candidates, and projects viewed. From here, companies navigate to manage their opportunity listings, review applicants, and browse student projects.



III - Expert Dashboard

The Expert Dashboard shows the reviewer’s workload at a glance: assigned reviews waiting to be scored, completed reviews, and pending feedback submissions. Experts navigate to their review queue and submission history from the top navigation.



IV - Admin / Mentor Dashboard

The Admin/Mentor Dashboard provides a real-time platform overview. The stat cards show total users, projects, pending reviews, and teams. Below, two panels list recent users (with roles) and recent projects (with statuses), giving mentors and admins immediate visibility into platform activity without navigating away.

